

Ask anything

To get the filter coefficients, you need to group the input terms x and output terms y separately.

By standardizing the equation, the **feedforward (b) coefficients** come from the input terms, and the **feedback (a) coefficients** come from the output terms.

1. Standardize the Equation

First, simplify the notation by replacing nT with n (so $nT - T$ becomes $n - 1$). Then, move all y terms to the left side and keep x terms on the right side:

$$y[n] - 2y[n-1] + y[n-2] = x[n] - 2x[n-6] + x[n-12]$$

2. Extract the Coefficients

Filter coefficients must include every consecutive delay step. If a delay step is missing from the equation, its coefficient is **0**.

- **Numerator Coefficients (b):** These match the x terms on the right side, starting from $x[n]$ up to $x[n-12]$.
- **Denominator Coefficients (a):** These match the y terms on the left side, starting from $y[n]$ up to $y[n-2]$.

3. Final Coefficient Arrays

When using software like MATLAB or Python (`scipy.signal`), you define the filter using these two arrays:

- b **(Feedforward):** `[1, 0, 0, 0, 0, 0, -2, 0, 0, 0, 0, 0, 1]`
  - $b_0 = 1$ (for $x[n]$)
  - b_1 to $b_5 = 0$



○ (for)

If you need to analyze this filter further, would you like to find its **frequency response**, plot its **pole-zero map**, or write the code to **implement it in Python/ MATLAB**?

Could a script written in python, withe use of sympy and/or scipy, take an arbitrary difference equation in text form an...

Yes, a **Python script using `sympy` and `scipy` can parse an arbitrary difference equation** from plain text and automatically extract the filter coefficients.

To achieve this reliably, the script can use **`sympy`** to handle the algebraic parsing (solving for and finding the coefficient of each delayed term) and then map those into the standard array format used by **`scipy.signal`**.

Here is a complete, working script that takes a text-based difference equation, parses it, and outputs the exact and coefficient arrays.

Python Implementation

python

```python
import re
from scipy import signal
import sympy as sp
```

```
# 3. Safely parse the equation text into a SymPy expression
# Assumes format "Left_Side = Right_Side"
lhs_str, rhs_str = eq_str.split("=")
lhs_expr = sp.sympify(lhs_str, locals={"x": x, "y": y, "n": n}
rhs_expr = sp.sympify(rhs_str, locals={"x": x, "y": y, "n": n}

# Bring everything to the standard form: LHS - RHS = 0
full_expr = sp.expand(lhs_expr - rhs_expr)

# 4. Find all present delays to determine array sizes
x_delays = []
y_delays = []

for arg in full_expr.atoms(sp.AppliedUndef):
    func_name = arg.name
    # Extract the shift value inside function, e.g., n - 2 ->
    shift_expr = arg.args[0] - n
    shift = int(shift_expr) if shift_expr != 0 else 0

    if func_name == "x":
        x_delays.append(-shift)  # we want positive indices fo
    elif func_name == "y":
        y_delays.append(-shift)

max_b_delay = max(x_delays) if x_delays else 0
max_a_delay = max(y_delays) if y_delays else 0

# 5. Extract coefficients sequentially into arrays
# In standard form (LHS): a terms are coefficients of y, b ter
b_coeffs = []
for i in range(max_b_delay + 1):
    term = x(n - i)
    # b coefficients are on the RHS, so they keep their sign r
    coeff = full_expr.coeff(term)
    b_coeffs.append(float(-coeff))
```

```
    return b, a


# --- Example Usage ---
equation_text = "y(nT) = 2*y(nT-T) - y(nT-2T) + x(nT) - 2*x(nT-6T)

b, a = extract_filter_coefficients(equation_text)

print(f"Feedforward coefficients (b): {b}")
print(f"Feedback coefficients (a):    {a}")

# Now you can plug them directly into scipy.signal
# w, h = signal.freqz(b, a)
```

Use code with caution.

How it works:

1. **Regex Normalization:** It converts dynamic text strings like `y(nT-2T)` or `y(nT - 2T)` safely into `y[n-2]`, which SymPy can recognize as discrete function steps.
2. **Algebraic Isolation:** It sets the entire equation to equal zero ( ).
3. **Dynamic Boundary Detection:** It scans the expression to find the highest delayed terms for both and so it knows exactly how long the arrays need to be.
4. **Zero-Padding:** It loops sequentially from delay `0` to the max delay. If `.coeff()` doesn't find a term (like `x[n-1]`), SymPy returns `0`, seamlessly populating missing steps.

Would you like me to expand this script to include a **visualization feature** (such

Here is the updated script. It combines the text parser with `scipy.signal` and `matplotlib` to automatically calculate and **plot the frequency response (magnitude and phase) and the pole-zero map** of the parsed filter.

python

```python
import sympy as sp
import re
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

def extract_filter_coefficients(equation_str):
    """Parses text equation and extracts standard b and a filter a
    # 1. Standardize text format
    eq_clean = equation_str.replace('=', '-(') + ')'
    eq_clean = re.sub(r'([xy])\s*[\(\[]\s*n(?:T)?\s*-\s*(\d+)(?:T)
    eq_clean = re.sub(r'([xy])\s*[\(\[]\s*n(?:T)?\s*[\)\]]', r'\1_

    # 2. Extract with SymPy
    var_names = set(re.findall(r'[xy]_\d+', eq_clean))
    symbols = {name: sp.Symbol(name) for name in var_names}
    expr = sp.sympify(eq_clean, locals=symbols)
    expr = sp.expand(expr)

    x_dict, y_dict = {}, {}
    for name, sym in symbols.items():
        coeff = expr.coeff(sym)
        signal_type, delay = name.split('_')
        delay = int(delay)

        if signal_type == 'x':
            x_dict[delay] = -float(coeff)
        elif signal_type == 'y':
```

```
        b = [x / a_0 for x in b]
        a = [x / a_0 for x in a]

    return b, a

def visualize_filter(b, a):
    """Generates Frequency Response and Pole-Zero plots for the fi
    # Compute Frequency Response
    w, h = signal.freqz(b, a, worN=2000)

    # Compute Poles and Zeros
    zeros, poles, k = signal.tf2zpk(b, a)

    # Create Layout (2 rows, 2 columns; bottom right stays empty c
    fig = plt.figure(figsize=(12, 10))

    # 1. Magnitude Response Plot
    ax1 = fig.add_subplot(2, 2, 1)
    ax1.plot(w / np.pi, 20 * np.log10(np.abs(h) + 1e-12), 'b')
    ax1.set_title("Magnitude Response")
    ax1.set_xlabel("Normalized Frequency (×π rad/sample)")
    ax1.set_ylabel("Amplitude (dB)")
    ax1.grid(True)

    # 2. Phase Response Plot
    ax2 = fig.add_subplot(2, 2, 2)
    ax2.plot(w / np.pi, np.angle(h), 'g')
    ax2.set_title("Phase Response")
    ax2.set_xlabel("Normalized Frequency (×π rad/sample)")
    ax2.set_ylabel("Phase (radians)")
    ax2.grid(True)

    # 3. Pole-Zero Plot
    ax3 = fig.add_subplot(2, 2, 3)
    # Draw Unit Circle
```

```
    ax3.set_ylabel("Imaginary Part")
    ax3.grid(True)
    ax3.axhline(0, color='black', lw=0.5)
    ax3.axvline(0, color='black', lw=0.5)
    ax3.axis('equal')
    ax3.legend()

    # 4. Info Panel
    ax4 = fig.add_subplot(2, 2, 4)
    ax4.axis('off')
    info_text = (
        f"Filter Summary:\n\n"
        f"Num of Coefficients (b): {len(b)}\n"
        f"Den of Coefficients (a): {len(a)}\n"
        f"Number of Zeros: {len(zeros)}\n"
        f"Number of Poles: {len(poles)}\n\n"
        f"System Status: {stability_text.upper()}"
    )
    ax4.text(0.1, 0.5, info_text, fontsize=12, family='monospace',

    plt.tight_layout()
    plt.show()

# --- Execution ---
equation = "y(nT) = 2*y(nT-T) - y(nT-2T) + x(nT) - 2*x(nT-6T) + x(
b, a = extract_filter_coefficients(equation)

print(f"b: {b}")
print(f"a: {a}")

# Call the visualization function
visualize_filter(b, a)
```

Use code with caution.

- **Stability Evaluation:** Automatically evaluates if any poles lie outside the unit circle ($|p| \geq 1$) to mark the filter as `Stable` or `UNSTABLE` on the plot text. (Note: Your specific example acts as an oscillator/comb-filter variant with poles exactly on the unit boundary line).

Would you like to test this script on a different system structure, such as an **IIR Butterworth filter**, or add an **impulse response visualization (time-domain)** to the dashboard?